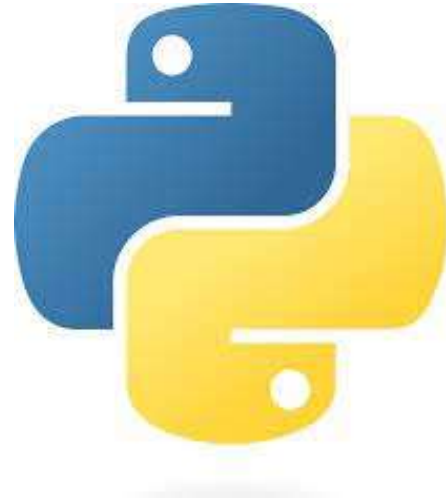




Python Lists

Python Lists

- A list is a collection which is ordered and changeable. In Python lists are written with square brackets.

Example:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

Output:

```
['apple', 'banana', 'cherry']
```

Python Lists

- Example: Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[1])
```

Output:
banana

- Negative indexing means beginning from the end, -1 refers to the last item, -2 refers to the second last item etc.

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[-1]) # output: cherry
```

Python Lists

- You can specify a range of indexes by specifying where to start and where to end the range.
- When specifying a range, the return value will be a new list with the specified items.

Example :

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon",  
            "mango"]  
print(thislist[2:5])
```

Output : ['cherry', 'orange', 'kiwi']

The search will start at index 2 (included) and end at index 5 (not included).

Python Lists

Example : This example returns the items from the beginning to "orange"

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon",  
            "mango"]  
print(thislist[:4])
```

#output : ['apple', 'banana', 'cherry', 'orange']

Example : This example returns the items from "cherry" and to the end:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon",  
            "mango"]  
print(thislist[2:])
```

#output : ['cherry', 'orange', 'kiwi', 'melon', 'mango']

Python Lists

- Specify negative indexes if you want to start the search from the end of the list:

Example : This example returns the items from index -4 (included) to index -1 (excluded)

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon",  
            "mango"]  
print(thislist[-4:-1])
```

#output : ['orange', 'kiwi', 'melon']

Python Lists

- To change the value of a specific item, refer to the index number:

Example : Change the second item:

```
thislist = ["apple", "banana", "cherry"]  
thislist[1] = "blackcurrant"  
print(thislist)
```

#output : ['apple', 'blackcurrant', 'cherry']

Python Lists

- Print all items in the list, one by one:

Example :

```
thislist = ["apple", "banana", "cherry"]  
for x in thislist:  
    print(x)
```

#output : apple
 banana
 cherry

Python Lists

- Check if "apple" is present in the list::

Example :

```
thislist = ["apple", "banana", "cherry"]  
if "apple" in thislist:  
    print("Yes, 'apple' is in the fruits list")
```

#output : Yes, 'apple' is in the fruits list

Python Lists

- Print the number of items in the list:

Example :

```
thislist = ["apple", "banana", "cherry"]  
print(len(thislist))
```

#output : 3

Python Lists

- To add an item to the end of the list, use the `append()` method:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
thislist.append("orange")  
print(thislist)
```

#output : ['apple', 'banana', 'cherry', 'orange']

Python Lists

- To add an item at the specified index, use the insert() method:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
thislist.insert(1, "orange")  
print(thislist)
```

#output : ['apple', 'orange', 'banana', 'cherry']

Python Lists

- The remove() method removes the specified item:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
thislist.remove("banana")  
print(thislist)
```

```
#output : ['apple', 'cherry']
```

Python Lists

- The pop() method removes the specified index, (or the last item if index is not specified):
- Example :

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop()  
print(thislist)
```

#output : ['apple', 'banana']

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop(1) #remove let at 1  
print(thislist)
```

#output : ['apple', "cherry"]

Python Lists

- The del keyword removes the specified index:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
del thislist[0]  
print(thislist)
```

#output : ['banana', 'cherry']

Python Lists

- The del keyword can also delete the list completely:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
del thislist  
print(thislist)
```

#output : Error

Python Lists

- The clear() method empties the list:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
thislist.clear()  
print(thislist)
```

#output : []

Python Lists

- Make a copy of a list with the copy() method:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
mylist = thislist.copy()  
print(mylist)
```

#output : ['apple', 'banana', 'cherry']

Python Lists

- Make a copy of a list with the list() method:
- Example :

```
thislist = ["apple", "banana", "cherry"]  
mylist = list(thislist)  
print(mylist)
```

#output : ['apple', 'banana', 'cherry']

Python Lists

- Join two list with +
- Example :

```
list1 = ["a", "b" , "c"]  
list2 = [1, 2, 3]  
list3 = list1 + list2  
print(list3)
```

#output : ['a', 'b', 'c', 1, 2, 3]

Python Lists

- Append list2 into list1: (same as +)
- Example :

```
list1 = ["a", "b" , "c"]  
list2 = [1, 2, 3]
```

```
for x in list2:  
    list1.append(x)
```

```
print(list1)
```

```
#output : ['a', 'b', 'c', 1, 2, 3]
```

Python Lists

- Use the extend() method to add list2 at the end of list1: (same as append)
- Example :

```
list1 = ["a", "b" , "c"]  
list2 = [1, 2, 3]  
list1.extend(list2)  
print(list1)
```

#output : ['a', 'b', 'c', 1, 2, 3]

Python Lists

- It is also possible to use the list() constructor to make a new list.
- Example 1 :

```
thislist = list(("apple", "banana", "cherry")) # note the double  
round-brackets  
print(thislist)
```

#output : ['apple', 'banana', 'cherry']

Ex2:

list("hello")

['h','e','l','l','o']

Python Lists

- Sorting the list

Syntax : Syntax

list.sort(reverse=True|False, key=myFunc)

reverse (Optional) : reverse=True will sort the list descending. Default is reverse=False

key (optional) : A function to specify the sorting criteria(s)

Python Lists

- Use `sort()` to sort the list
- Example :

```
mylist = ["banana", "cherry", "apple"]  
mylist.sort()  
print(mylist)
```

#output : ['apple', 'banana', 'cherry']

Python Lists

- Use `sort(reverse=True)` to sort the list in reverse order.
- Example :

```
cars = ['Ford', 'BMW', 'Volvo']  
cars.sort(reverse=True)  
Print(cars)
```

```
#output : ['Volvo', 'Ford', 'BMW']
```

Python Lists

- You can Sort the list by the length of the values:

Example :

```
# A function that returns the length of the value:
```

```
def myFunc(e):  
    return len(e)
```

```
cars = ['Ford', 'Mitsubishi', 'BMW', 'VW']
```

```
cars.sort(key=myFunc)
```

Python Lists

- Python has a set of built-in methods that you can use on lists.

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position

Python Lists

pop() Removes the element at the specified position

remove() Removes the item with the specified value

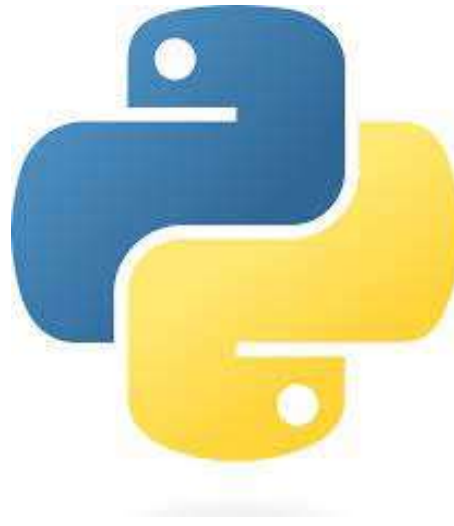
reverse() Reverses the order of the list

sort() Sorts the list

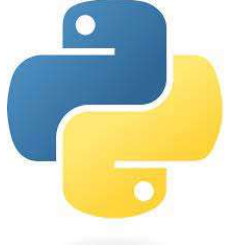
count()

```
nums = [1, 2, 3, 2, 4, 2, 5]
```

```
print(nums.count(2))
```



Python Strings



Python Strings

- **String literal**
- **Multiline strings**
- **Accessing Elements**
- **Membership operator with strings**
- **Using + and * with strings**
- **Using Format()**
- **Escape Characters**
- **String methods**

Python Strings – String literal

- String literals in python are surrounded by either single quotation marks, or double quotation marks.

'hello' is the same as "hello"

- Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

```
a = "Hello"  
print(a)
```


Python Strings - multiline strings

- You can assign a multiline string to a variable by using three quotes (double or single):

Example-1

```
a = """Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua."""
```

```
print(a)
```

Note: in the result, the line breaks are inserted at the same position as in the code.

Python Strings - multiline strings

- Following example shows multiline string surrounded by triple single quotes

Example-2

```
a = "Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua"
```

```
print(a)
```

Python Strings - Accessing Elements

- Strings in Python are arrays of bytes representing unicode characters.
- However, Python does not have a character data type, a single character is simply a string with a length of 1
- Square brackets can be used to access elements of the string.

Example

```
a = "Hello, World!"  
print(a[1])
```

Output:

e

Python Strings - Accessing Elements

- You can return a range of characters by using **string slicing**
- Specify the start index and the end index, separated by a colon, to return a part of the string.

Example :Get the characters from position 2 to position 5 (not included):

```
b = "Hello, World!"  
print(b[2:5])
```

Output :
llo

•

Python Strings - Accessing Elements

- The term 'string slice' refers to a part of the string, where strings are sliced using a range of indices. For a string say name[n : m] where n and m are integers and legal indices. Python will return a slice of the string by returning the characters falling between indices n and m starting at n, n+1, n+2... till m-1.

Let's understand Slicing in strings with the help of few examples.

String word	a	m	a	z	i	n	g
Positive Index	0	1	2	3	4	5	6
Negative Index	-7	-6	-5	-4	-3	-2	-1

Python Strings - Accessing Elements

- Slice From the Start

Get the characters from the start to position 5 (not included):

```
b = "Hello, World!"  
print(b[:5])
```

.

Output:
Hello

Python Strings - Accessing Elements

- Slice To the End

Get the characters from position 2, and all the way to the end:

```
b = "Hello, World!"  
print(b[2:])
```

Output:
llo, World!

Python Strings - Accessing Elements

- **Negative Indexing** : Use negative indexes to start the slice from the end of the string: **(Count from -1 from the end for negative index)**

Example : Get the characters from position -5 to position -2 (not included)

```
b = "Hello, World!"  
print(b[-5:-2])
```

Output :
orl

Python Strings - Accessing Elements

- More Example for string slicing :

```
x = "Hello, Python"  
print(x[-5:])
```

Output:
ython

```
print(x[:-3])
```

Output:
Hello, Pyt

```
print(x[7:-1])
```

Output:
Pytho

```
print(x[:])
```

Output:
Hello, Python

Python Strings - Accessing Elements

- You can also include a third number in slicing to indicate skipping step. In the following example, we start from index 1 and end with index 6 index(stops at $6-1=5$), and the skipping step is 2. It is a good example of Python slicing string by character.

```
x = "Hello, Python"  
print(x[1:6:2])
```

Output:
el,

Python Strings - Accessing Elements

- Also when skipping step is negative, string slicing is reversed. In this case first number indicates start index in the reverse order, second number is end index (not included) in the reverse order, and the last number is skipping step in the reverse order.

- Examples :

```
x = "Hello, Python"  
print(x[4:1:-1])
```

Output :
oll

Python Strings - Accessing Elements

`string[start : stop : step]`

Examples :

```
x = "Hello, Python"
```

```
print(x[5:0:-2])
```

(-2 is moving towards backard)

```
print(x[8: : -2])
```

```
print(x[: 6: -2])
```

```
print(x[ : :-1])
```

Output :
le,

Output :
H ol y

Output :
yhn

Output :
H olle ,Pytno

Python Strings - Accessing Elements

- Examples :

```
x = "Hello, Python"  
print(x[-2:7:-1])
```

```
print(x[4:-12:-1])
```

```
print(x[-2:-6:-1])
```

Output :
ohty

Output :
oll

Output :
ohty

Python Strings – Getting String Length

- To get the length of a string, use the `len()` function

Example :

```
a = "Hello, World!"  
print(len(a))
```

Output :
13

Python string - membership operator

- To check if a certain phrase or character is present in a string, we can use the keywords **in** or **not in**.

Example : Check if the phrase "ain" is present in the following text:

```
txt = "The rain in Spain stays mainly in the plain"  
x = "ain" in txt  
print(x)  
x = "ain" not in txt  
print(x)
```

Output:
True
False

Python **Strings** – using +

- To concatenate, or combine, two strings you can use the + operator.
- Example :

```
a = "Hello"  
b = "World"  
c = a + b  
print(c)
```

Output:
HelloWorld

Python **Strings** - using +

Example :

To add a space between them, add a " ":

```
a = "Hello"  
b = "World"  
c = a + " " + b  
print(c)
```

Output:
Hello World

Python **Strings** - using *****

- (Replication) operator: The ***** operator replicates the string the number of times the value given on the other side.
- Consider the following example `>>>3*"go "`
`"gogogo"`

Python **Strings** - **format()**

- The `format()` method takes unlimited number of arguments, and are placed into the respective placeholders:

Example :

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want {} pieces of item {} for {} dollars."
print(myorder.format(quantity, itemno, price))
```

Output :

I want 3 pieces of item 567 for 49.95 dollars.

Python **Strings** - **format()**

- You can use index numbers inside {} to make sure that arguments are placed in the correct placeholders:

Example :

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want to pay {2} dollars for {0} pieces of
item {1}."
print(myorder.format(quantity, itemno, price))
```

Output :

I want to pay 49.95 dollars for 3 pieces of item 567

Python escape character

- To insert characters that are illegal in a string, use an **escape** character.
- An escape character is a backslash \ followed by the character you want to insert.

Example :

```
txt = "We are the so-called \"Vikings\" from the  
north."
```

```
Print(txt)
```

Output:

We are the so-called "Vikings" from the north.

Python escape character

- Other escape characters used in Python:

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

Python String Formatting Operator

- One of Python's coolest features is the string format operator %.
- This operator is unique to strings and makes up for the pack of having functions from C's printf() family.
Following is a simple example -

```
print "My name is %s and weight is %d kg!" % ('Zara', 21)
```

Output : My name is Zara and weight is 21 kg!

Python String Formatting Operator

- Here is the list of complete set of symbols which can be used along with `%`

Format Symbol	Conversion
<code>%c</code>	character
<code>%s</code>	string conversion via <code>str()</code> prior to formatting
<code>%i</code>	signed decimal integer
<code>%d</code>	signed decimal integer
<code>%u</code>	unsigned decimal integer
<code>%o</code>	octal integer
<code>%x</code>	hexadecimal integer (lowercase letters)
<code>%X</code>	hexadecimal integer (UPPERcase letters)

Python String Formatting Operator

- Here is the list of complete set of symbols which can be used along with % –

%e	exponential notation (with lowercase 'e')
%E	exponential notation (with UPPERCASE 'E')
%f	floating point real number
%g	the shorter of %f and %e
%G	the shorter of %f and %E

Python String - methods

- The `strip()` method removes any whitespace from the beginning or the end:

Example :

```
a = " Hello, World! "  
print(a.strip()) # returns "Hello, World!"
```

Python String - methods

- The `lower()` and `upper()` method returns the string in lower and upper case respectively:

Example :

```
a = "Hello, World!"  
print(a.lower())  
print(a.upper())
```

Output :

```
hello, world!  
HELLO, WORLD!
```

Python String – methods

- The `replace()` method replaces a string with another string:

Example :

```
a = "Hello, World!"  
print(a.replace("H", "J"))
```

Output:
Jello, World!

Python String – methods

- The `split()` method splits the string into substrings if it finds instances of the separator:

Example :

```
a = "Hello, World!"  
print(a.split(",")) # returns ['Hello', ' World!']
```

Output:
['Hello', ' World!']

Python String - All methods

Method	Description
<u>capitalize()</u>	Converts the first character to upper case
<u>casefold()</u>	Converts string into lower case
<u>center()</u>	Returns a centered string
<u>count()</u>	Returns the number of times a specified value occurs in a string
<u>encode()</u>	Returns an encoded version of the string
<u>endswith()</u>	Returns true if the string ends with the specified value

Python String - All methods

expandtabs() Sets the tab size of the string

find() Searches the string for a specified value and returns the position of where it was found

format() Formats specified values in a string

format_map() Formats specified values in a string

index() Searches the string for a specified value and returns the position of where it was found

isalnum() Returns True if all characters in the string are alphanumeric

isalpha() Returns True if all characters in the string are in the alphabet

isascii() Returns True if all characters in the string are ascii characters

Python String - All methods

isdecimal() Returns True if all characters in the string are decimals

isdigit() Returns True if all characters in the string are digits

isidentifier() Returns True if the string is an identifier

islower() Returns True if all characters in the string are lower case

isnumeric() Returns True if all characters in the string are numeric

isprintable() Returns True if all characters in the string are printable

isspace() Returns True if all characters in the string are whitespaces

istitle() Returns True if the string follows the rules of a title

Python String - All methods

<u>isupper()</u>	Returns True if all characters in the string are upper case
------------------	---

<u>join()</u>	Converts the elements of an iterable into a string
---------------	--

<u>ljust()</u>	Returns a left justified version of the string
----------------	--

<u>lower()</u>	Converts a string into lower case
----------------	-----------------------------------

<u>lstrip()</u>	Returns a left trim version of the string
-----------------	---

<u>maketrans()</u>	Returns a translation table to be used in translations
--------------------	--

<u>partition()</u>	Returns a tuple where the string is parted into three parts
--------------------	---

Python String - All methods

<u>rindex()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rjust()</u>	Returns a right justified version of the string
<u>rpartition()</u>	Returns a tuple where the string is parted into three parts
<u>rsplit()</u>	Splits the string at the specified separator, and returns a list
<u>rstrip()</u>	Returns a right trim version of the string
<u>split()</u>	Splits the string at the specified separator, and returns a list
<u>splitlines()</u>	Splits the string at line breaks and returns a list
<u>startswith()</u>	Returns true if the string starts with the specified value

Python String - All methods

strip() Returns a trimmed version of the string

swapcase() Swaps cases, lower case becomes upper case and vice versa

title() Converts the first character of each word to upper case

translate() Returns a translated string

upper() Converts a string into upper case

zfill() Fills the string with a specified number of 0 values at the beginning



NITTE
(Deemed to be University)

**NMAM INSTITUTE
OF TECHNOLOGY**

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

Introduction To Python Programming

Course Outcomes:

At the end of the course student will be able to

1. Understand and experiment with the basics of python programming like data types and looping.
2. Describe the concept of functions in python for solving real-world problems.
3. Apply the Python operations for manipulating strings, lists, tuples and dictionaries.
4. Demonstrate the proficiency in handling File Systems.
5. Understand the basic knowledge on Exception handling and plotting graphs.

Textbooks:

1. Kenneth A. Lambert, “The Fundamentals of Python: First Programs”, Cengage Learning, 2nd Edition, 2019.
2. Yashwanth Kanetkar, Adithya Kanetkar, “Let us Python”, 3rd Edition.
3. Mark Summerfield, “Programming in Python 3 - A Complete Introduction to the Python Language”, Second Edition, Addison-Wesley, 2009.
4. Y. Daniel Liang, “Introduction to Programming Using Python”, Pearson, 2013.

Tuples , Sets, Dictionaries

Tuples

- A tuple is same as list, except that the set of elements is enclosed **in parentheses** instead of square brackets.
- A tuple is an **immutable** list. i.e. once a tuple has been created, you can't add elements to a tuple or remove elements from the tuple.
- But tuple can be converted into list and list can be converted into tuple.

methods	example	description
list()	<pre>>>> a=(1,2,3,4,5) >>> a=list(a) >>> print(a) [1, 2, 3, 4, 5]</pre>	it convert the given tuple into list.
tuple()	<pre>>>> a=[1,2,3,4,5] >>> a=tuple(a) >>> print(a) (1, 2, 3, 4, 5)</pre>	it convert the given list into tuple.

Benefit of Tuple:

- Tuples are faster than lists.
- If the user wants to protect the data from accidental changes, tuple can be used.
- Tuples can be used as keys in dictionaries, while lists can't.

Operations on Tuples:

1. Indexing
2. Slicing
3. Concatenation
4. Repetitions
5. Membership
6. Comparison

Operations	examples	description
Creating a tuple	>>>a=(20,40,60,"apple","ball")	Creating the tuple with elements of different data types.
Indexing	>>>print(a[0]) 20 >>> a[2] 60	Accessing the item in the position 0 Accessing the item in the position 2
Slicing	>>>print(a[1:3]) (40,60)	Displaying items from 1st till 2nd.
Concatenation	>>> b=(2,4) >>>print(a+b) >>>(20,40,60,"apple","ball",2,4)	Adding tuple elements at the end of another tuple elements
Repetition	>>>print(b*2) >>>(2,4,2,4)	repeating the tuple in n no of times
Membership	>>> a=(2,3,4,5,6,7,8,9,10) >>> 5 in a True >>> 100 in a False >>> 2 not in a False	Returns True if element is present in tuple. Otherwise returns false.
Comparison	>>> a=(2,3,4,5,6,7,8,9,10) >>>b=(2,3,4) >>> a==b False >>> a!=b True	Returns True if all elements in both elements are same. Otherwise returns false

Tuple methods:

- Tuple is immutable so changes cannot be done on the elements of a tuple once it is assigned.

methods	example	description
a.index(tuple)	>>> a=(1,2,3,4,5) >>> a.index(5) 4	Returns the index of the first matched item.
a.count(tuple)	>>>a=(1,2,3,4,5) >>> a.count(3) 1	Returns the count of the given element.
len(tuple)	>>> len(a) 5	return the length of the tuple
min(tuple)	>>> min(a) 1	return the minimum element in a tuple
max(tuple)	>>> max(a) 5	return the maximum element in a tuple
del(tuple)	>>> del(a)	Delete the entire tuple.

Tuple Assignment:

- Tuple assignment allows, variables on the left of an assignment operator and values of tuple on the right of the assignment operator.
- Multiple assignment works by creating a tuple of expressions from the right hand side, and a tuple of targets from the left, and then matching each expression to a target.
- Because multiple assignments use tuples to work, it is often termed tuple assignment.

Uses of Tuple assignment:

- It is often useful to swap the values of two variables.
- Example:
- Swapping using temporary variable:

a=20

b=50

temp = a

a = b

b = temp

print("value after swapping is",a,b)

Swapping using tuple assignment:

a=20

b=50

(a,b)=(b,a)

print("value after swapping is",a,b)

Multiple assignments:

- Multiple values can be assigned to multiple variables using tuple assignment.

```
>>>(a,b,c)=(1,2,3)
```

```
>>>print(a)
```

```
1
```

```
>>>print(b)
```

```
2
```

```
>>>print(c)
```

```
3
```


Tuple as return value:

- A Tuple is a comma separated sequence of items.
- It is created with or without ().
- A function can return one value. if you want to return more than one value from a function. we can use tuple as return value.

Eg:~

```
def div(a,b):  
    r=a%b  
    q=a//b  
    return(r,q)  
a=int(input("enter a value:"))  
b=int(input("enter b value:"))  
r,q=div(a,b)  
print("reminder:",r)  
print("quotient:",q)
```

Output:

```
enter a value:4  
enter b value:3  
reminder: 1  
quotient: 1
```

Eg:2

```
def minmax(a):  
    small=min(a)  
    big=max(a)  
    return(small, big)  
a=[1,2,3,4,6]  
small, big=minmax(a)  
print("smallest:", small)  
print("biggest:", big)
```

Output:

smallest: 1

biggest: 6

Sets

Sets

- A Set in Python is used to store a collection of items with the following properties.
- 1)No duplicate elements. If try to insert the same item again, it overwrites previous one.
- 2)An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.

Unchangeable

Set items are unchangeable, meaning that we cannot change the items after the set has been created.

```
s = {10, 50, 20}
```

```
print(s)
```

```
print(type(s))
```

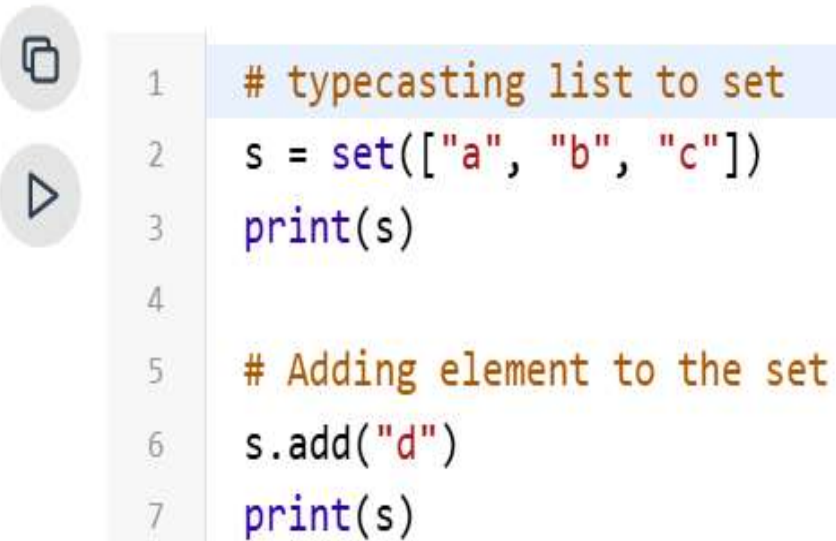
Output

```
{10, 50, 20}
```

```
<class 'set'>
```

Type Casting with Python Set method

The Python set() method is used for type casting.



```
1 # typecasting list to set
2 s = set(["a", "b", "c"])
3 print(s)
4
5 # Adding element to the set
6 s.add("d")
7 print(s)
```

Output

```
{'c', 'b', 'a'}
{'d', 'c', 'b', 'a'}
```

Accessing Set Elements

```
thisset = {"apple", "banana", "cherry"}
```

```
for x in thisset:  
    print(x)
```


Python - Add Set Items

- Add an item to a set, using the add() method:

Add an item to a set, using the add() method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.add("orange")  
  
print(thisset)
```

Add Sets

Example

Add elements from `tropical` into `thisset`:

```
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}  
  
thisset.update(tropical)  
  
print(thisset)
```

Add Any Iterable

- The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

Example

Add elements of a list to a set:

```
thisset = {"apple", "banana", "cherry"}  
mylist = ["kiwi", "orange"]  
  
thisset.update(mylist)  
  
print(thisset)
```

Python - Remove Set Items

Remove Item

- To remove an item in a set, use the `remove()`, or the `discard()` method. (If the item to remove does not exist, `remove()` will raise an error)

```
thisSet={"Priya","joylin","pinto","NMAMIT"}  
thisSet.remove("joylin")  
print(thisSet)
```

Output: {'NMAMIT', 'Priya', 'pinto'}

```
thisSet={"Priya","joylin","pinto","NMAMIT"}
```

```
thisSet.remove("Nitte")
```

```
Output: {'NMAMIT', 'Priya', 'pinto'}
```

Traceback (most recent call last):

File "C:/Users/Priya/Desktop/2025-26/EVEN/Introduction to Python Programming (2025-26)/Example Programs/Set/test.py", line 2, in
<module>

```
    thisSet.remove("Nitte")
```

```
KeyError: 'Nitte'
```

- Remove “Nitte” by using the discard() method: If the item to remove does not exist, discard() will NOT raise an error.

```
thisSet={"Priya","joylin","pinto","NMAMIT"}
```

```
thisSet.remove("Nitte")
```

```
print(thisSet)
```

- You can also use the `pop()` method to remove an item, but this method will remove a random item, so you cannot be sure what item that gets removed.
- The return value of the `pop()` method is the removed item.

Remove a random item by using the `pop()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
x = thisset.pop()  
  
print(x)  
  
print(thisset)
```

- The `clear()` method empties the set:
- `thisset.clear()`

The `del` keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}
```

```
del thisset
```

```
print(thisset)
```


Join Sets

- There are several ways to join two or more sets in Python.
- The `union()` and `update()` methods joins all items from both sets.
- The `intersection()` method keeps ONLY the duplicates.
- The `difference()` method keeps the items from the first set that are not in the other set(s).
- The `symmetric_difference()` method keeps all items EXCEPT the duplicates.

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = set1.union(set2)
```

```
print(set3)
```

```
myset = set1.union(set2, set3, set4)
```

Use | to join two sets:

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = set1 | set2
```

```
print(set3)
```

```
myset = set1 | set2 | set3 | set4
```

Dictionaries

- Dictionary is an unordered collection of elements.
- An element in dictionary has a key: value pair.
- All elements in dictionary are placed inside the curly braces i.e. { }
- Elements in Dictionaries are **accessed via keys** and not by their position.
- The values of a dictionary can be any data type.
- Keys must be immutable data type (numbers, strings, tuple)

Operations on dictionary:

1. Accessing an element
2. Update
3. Add element
4. Membership

Operations	Example	Description
Creating a dictionary	<pre>>>> a={1:"one",2:"two"} >>> print(a) {1: 'one', 2: 'two'}</pre>	Creating the dictionary with elements of different data types.
accessing an element	<pre>>>> a[1] 'one' >>> a[0] KeyError: 0</pre>	Accessing the elements by using keys.
Update	<pre>>>> a[1]="ONE" >>> print(a) {1: 'ONE', 2: 'two'}</pre>	Assigning a new value to key. It replaces the old value by new value.
add element	<pre>>>> a[3]="three" >>> print(a) {1: 'ONE', 2: 'two', 3: 'three'}</pre>	Add new element in to the dictionary with key.
membership	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> 1 in a True >>> 3 not in a False</pre>	Returns True if the key is present in dictionary. Otherwise returns false.

a.update(dictionary)	<pre>>>> b={4:"four"} >>> a.update(b) >>> print(a) {1: 'ONE', 2: 'two', 3: 'three', 4: 'four'}</pre>	It will add the dictionary with the existing dictionary
fromkeys()	<pre>>>> key={"apple","ball"} >>> value="for kids" >>> d=dict.fromkeys(key,value) >>> print(d) {'apple': 'for kids', 'ball': 'for kids'}</pre>	It creates a dictionary from key and values.
len(a)	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> len(a) 3</pre>	It returns the length of the list.
clear()	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> a.clear() >>> print(a) >>> {}</pre>	Remove all elements form the dictionary.
del(a)	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> del(a)</pre>	It will delete the entire dictionary.

Methods in dictionary:

Method	Example	Description
a.copy()	<pre>a={1: 'ONE', 2: 'two', 3: 'three'} >>> b=a.copy() >>> print(b) {1: 'ONE', 2: 'two', 3: 'three'}</pre>	It returns copy of the dictionary. here copy of dictionary 'a' get stored in to dictionary 'b'
a.items()	<pre>>>> a.items() dict_items([(1, 'ONE'), (2, 'two'), (3, 'three')])</pre>	Return a new view of the dictionary's items. It displays a list of dictionary's (key, value) tuple pairs.
a.keys()	<pre>>>> a.keys() dict_keys([1, 2, 3])</pre>	It displays list of keys in a dictionary
a.values()	<pre>>>> a.values() dict_values(['ONE', 'two', 'three'])</pre>	It displays list of values in dictionary
a.pop(key)	<pre>>>> a.pop(3) 'three' >>> print(a) {1: 'ONE', 2: 'two'}</pre>	Remove the element with <i>key</i> and return its value from the dictionary.
setdefault(key,value)	<pre>>>> a.setdefault(3,"three") 'three' >>> print(a) {1: 'ONE', 2: 'two', 3: 'three'} >>> a.setdefault(2) 'two'</pre>	If key is in the dictionary, return its value. If key is not present, insert key with a value of dictionary and return dictionary.

Difference between List, Tuples and dictionary:

List	Tuples	Dictionary
A list is mutable	A tuple is immutable	A dictionary is mutable
Lists are dynamic	Tuples are fixed size in nature	In values can be of any data type and can repeat, keys must be of immutable type
List are enclosed in brackets [] and their elements and size can be changed	Tuples are enclosed in parenthesis () and cannot be updated	Tuples are enclosed in curly braces { } and consist of key:value
Example: List = [10, 12, 15]	Example: Words = ("spam", "eggs") Or Words = "spam", "eggs"	Example: Dict = {"ram": 26, "abi": 24}
Access: print(list[0])	Access: print(words[0])	Access: print(dict["ram"])
Can contain duplicate elements	Can contain duplicate elements. Faster compared to lists	Cant contain duplicate keys, but can contain duplicate values
Slicing can be done	Slicing can be done	Slicing can't be done
Usage: ❖ List is used if a collection of data that doesn't need random access. ❖ List is used when data can be modified frequently	Usage: ❖ Tuple can be used when data cannot be changed. ❖ A tuple is used in combination with a dictionary i.e. a tuple might represent a key.	Usage: ❖ Dictionary is used when a logical association between key:value pair. ❖ When in need of fast lookup for data, based on a custom key. ❖ Dictionary is used when data is being constantly modified.